

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

```
1 #Type Content here...
2 m = float(input(""))
3 v = float(input(""))
4 p = m*v
5 print(f"{p:.2f}kgm/s")
```

Average time
0.020 s
19.75 ms

Maximum time
0.039 s
39.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 39 ms Debug

Expected output

Actual output

5.05.0

10.010.0

50.00kgm/s50.00kgm/s

✓ Test case 2 20 ms

Terminal

Test cases

1.1.2. Conditional Calculation Based on the Number of Digits 04:16

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases +

condition...

```
1 #write your code here...
2 import math
3 n = int(input(""))
4 if 0<n<10:
5     print(n*n)
6 elif 10<=n<=99:
7     print(f"{math.sqrt(n):.2f}")
8 elif 100<=n<=999:
9     print(f"{math.cbrt(n):.2f}")
10 else:
11     print("Invalid")
```

Average time
0.007 s
6.57 ms

Maximum time
0.010 s
10.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 10 ms Debug

Expected output
9
81

Actual output
9
81

Test case 2 7 ms

Test case 3 5 ms

Terminal

Test cases

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format `YYYY - MM - DD`.
- The second line contains the second date in the format `YYYY - MM - DD`.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noOfDay...

```
1 from datetime import date
2
3 #write your code here...
4 from datetime import datetime
5
6 date1=input()
7 date2=input()
8
9 d1=datetime.strptime(date1, "%Y-%m-%d")
10 d2=datetime.strptime(date2, "%Y-%m-%d")
11
```

Average time
0.019 s
19.00 ms

Maximum time
0.050 s
50.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 50 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 9 ms

Terminal

Test cases

< Prev Reset Submit Next >

1.1.4. Reverse a Number

01:06

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

+

reverseN...

Submit

```
1 #write your code here...
2 num = int(input(""))
3 n = int(str(num)[::-1])
4 print(n)
```

Average time

0.028 s

28.40 ms

Maximum time

0.040 s

40.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 40 ms

Debug

Expected output

5367

7635

Actual output

5367

7635

Test case 2 34 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

multiplica...

```
1 #write your code here...
2 n= int(input(""))
3 for i in range(1,11,1):
4     print(f"{n} x {i} = {n*i}")
5
```

Average time
0.009 s
9.25 ms

Maximum time
0.010 s
10.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 10 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

Terminal Test cases

< Prev

Reset

Submit

Next >

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

```
1 # write your code here# Accept number of courses
2 n = int(input())
3 # Accept marks as a list of integers
4 marks = list(map(int, input().split()))
5
6 # Check for failure in any course
7 if any(m < 40 for m in marks):
8     print("Fail")
9 else:
10     # Calculate aggregate percentage
11     aggregate_percentage = sum(marks) / n
```

Average time
0.015 s
15.30 ms

Maximum time
0.021 s
21.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 15 ms Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2 20 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.2. Fibonacci Series

43:22

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

+

fibonacci...

Submit

```
1 def fibonacci(n):
2
3     if n <= 0:
4         return 0
5     elif n <= 1:
6         return 1
7     return fibonacci(n-1) + fibonacci(n-2)
8
9
10 n = int(input())
11 # for i in range(1, n + 1):
```

Average time

0.019 s

19.25 ms

Maximum time

0.050 s

50.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms

Debug

Expected output

5

0 1 1 2 3

Actual output

5

0 1 1 2 3

Test case 2 7 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.3. Pattern - 114.03

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases+

rightangl...

```
1 n = int(input(""))
2 for i in range(1,n+1):
3     for j in range(0,i):
4         print("*",end=" ")
5     print()
```

Average time0.009 s8.83 msMaximum time0.010 s10.00 ms

2 out of 2 shown test case(s) passed4 out of 4 hidden test case(s) passed

Test case 110 msDebug

Expected output

5

*
* *
* * *
* * * *
* * * * *

Actual output

5

*
* *
* * *
* * * *
* * * * *

TerminalTest cases

< PrevResetSubmitNext >

1.2.4. Pattern - 202:59

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases+

numberP...

```
1 row = int(input(""))
2 n=1
3 for i in range(1,row+1):
4     for j in range(1,i+1):
5         print(j,end=" ")
6     print()
```

Average time0.006 s6.25 msMaximum time0.008 s8.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 17 msDebug

Expected output	Actual output
5	5
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

TerminalTest cases

< PrevResetSubmitNext >